

Université Versailles Saint-Quentin-en-Yvelines

Master CHPS

TP3 PPCS

SLURM, Docker Compose, exécution MPI et télémétrie
(InfluxDB/Chronograf)

Réalisé par :

Arab Lina

Année universitaire : 2025/2026

Date : Janvier 2026

Table des matières

1	Objectif du TP3	2
2	Déploiement de l'environnement Docker	2
2.1	Démarrage des services	2
2.2	Résolution d'un problème de réseau Docker	2
2.3	Accès aux interfaces Web	3
3	Configuration et validation de SLURM	3
3.1	Connexion au conteneur <code>slurmctld</code>	3
3.2	Vérification des associations, partition et nœuds	3
3.3	Synchronisation de la configuration sur les nœuds	3
4	Exécution d'un job MPI via SLURM	4
4.1	Récupération du dépôt de test	4
4.2	Script batch SLURM : <code>test-batch.sh</code>	4
4.3	Résultats SLURM	4
4.4	Résultat applicatif	4
4.5	Export des fichiers vers Windows	5
5	Télémétrie : StatsD, InfluxDB et Chronograf	5
5.1	Envoi d'une métrique StatsD	5
5.2	Visualisation dans Chronograf	5
6	Diagramme d'architecture du mini-cluster	6
6.1	Diagramme généré	6
6.2	Script PlantUML	6
7	Conclusion	8

1 Objectif du TP3

L'objectif de ce TP3 (module PPCS) est de :

- ▷ déployer une architecture conteneurisée via **Docker Compose** comprenant un ordonnanceur **SLURM** et des nœuds de calcul ;
- ▷ exécuter un job MPI distribué sur plusieurs nœuds via **sbatch** / **srun** ;
- ▷ mettre en place une chaîne de télémétrie **StatsD/Telegraf** vers **InfluxDB** et visualiser via **Chronograf** (et **Grafana**) ;
- ▷ produire des preuves de bon fonctionnement (sorties **SLURM**, sorties job, visualisation **Chronograf**).

Ce rapport décrit l'ensemble des actions effectuées, les commandes exécutées et les résultats obtenus, de A à Z.

2 Déploiement de l'environnement Docker

2.1 Démarrage des services

Le projet fournit un fichier `docker-compose.yml` permettant de lancer plusieurs services :

- ▷ **slurmctld** : contrôleur **SLURM** ;
- ▷ **slurmdbd** : service de base de données **SLURM** ;
- ▷ **mysql** : base de données pour **slurmdbd** ;
- ▷ **c1, c2, c3, c4** : nœuds de calcul (démons **slurmd**) ;
- ▷ **influxdb_local** : **InfluxDB** + **Telegraf** + **Chronograf/Grafana** (selon l'image fournie) ;
- ▷ **vscode** : code-server (IDE web).

Le démarrage est réalisé sur Windows (PowerShell) :

```
docker compose up -d
```

2.2 Résolution d'un problème de réseau Docker

Lors d'un premier démarrage, une erreur empêchait l'attachement des conteneurs au réseau :

```
failed to set up container networking:  
Could not attach to network yml_mpinet:  
network yml_mpinet not found
```

La correction consistait à créer le réseau Docker requis :

```
docker network create yml_mpinet
```

Puis relancer le démarrage :

```
docker compose up -d
```

2.3 Accès aux interfaces Web

Après démarrage, les interfaces suivantes étaient accessibles :

- ▷ Grafana : `http://127.0.0.1:3003`
- ▷ Chronograf : `http://127.0.0.1:3004`
- ▷ VSCode : `http://127.0.0.1:8081`
- ▷ InfluxDB : `http://127.0.0.1:8086`

3 Configuration et validation de SLURM

3.1 Connexion au conteneur slurmctld

Toutes les commandes SLURM suivantes sont exécutées dans le conteneur contrôleur :

```
docker exec -it slurmctld bash
```

3.2 Vérification des associations, partition et nœuds

Commandes utilisées :

```
sacctmgr show association -p user=root
scontrol show partition
scontrol show nodes
sinfo -Nel
```

3.3 Synchronisation de la configuration sur les nœuds

Initialement, SLURM ne déclarait que `c1` et `c2`. Afin d'obtenir un cluster de 4 nœuds (`c1..c4`), le fichier `/etc/slurm/slurm.conf` a été synchronisé sur l'ensemble des conteneurs `c1..c4`.

Méthode utilisée :

- ▷ copier `slurm.conf` depuis `slurmctld` vers l'hôte ;
- ▷ recopier le même fichier `slurm.conf` vers chacun des nœuds `c1..c4` ;
- ▷ redémarrer les conteneurs SLURM.

Commandes (PowerShell) :

```
docker cp slurmctld:/etc/slurm/slurm.conf ./slurm.conf
docker cp ./slurm.conf c1:/etc/slurm/slurm.conf
docker cp ./slurm.conf c2:/etc/slurm/slurm.conf
docker cp ./slurm.conf c3:/etc/slurm/slurm.conf
docker cp ./slurm.conf c4:/etc/slurm/slurm.conf
docker restart slurmctld c1 c2 c3 c4
```

Vérification :

```
sinfo -Nel
```

Résultat :

```
NODELIST NODES PARTITION STATE CPUS S:C:T MEMORY
c1 1 docker* idle 1 1:1:1 1000
c2 1 docker* idle 1 1:1:1 1000
c3 1 docker* idle 1 1:1:1 1000
c4 1 docker* idle 1 1:1:1 1000
```

4 Exécution d'un job MPI via SLURM

4.1 Récupération du dépôt de test

Depuis `slurmctld`, dans `/usr/local/var/mpishare` :

```
cd /usr/local/var/mpishare
git clone http://gogs.eldarsoft.com/M2_IHPS/glcs_slurm.git
```

4.2 Script batch SLURM : `test-batch.sh`

Le script `test-batch.sh` demande :

- ▷ 2 nœuds (`#SBATCH -nodes=2`);
- ▷ 2 tâches au total (`#SBATCH -ntasks=2`);
- ▷ 1 tâche par nœud (`#SBATCH -ntasks-per-node=1`);
- ▷ fichiers de sortie `res_%j.out` et erreurs `res_%j.err`.

Soumission :

```
cd /usr/local/var/mpishare/glcs_slurm
sbatch test-batch.sh
```

4.3 Résultats SLURM

Commandes :

```
squeue
sacct --format=JobID,JobName,Elapsed,NNodes,AllocCPUS,State,NodeList
```

Résultat obtenu :

```
JobID JobName Elapsed NNodes AllocCPUS State NodeList
1 test_mpi 00:00:02 2 2 COMPLETED c[1-2]
```

4.4 Résultat applicatif

Le fichier `res_1.out` contient :

```
=====
Job SLURM ID : 1
Date de dbut : Sun Jan 11 01:33:38 UTC 2026
Excut sur le noeud matre : c1
Liste des noeuds allous : c[1-2]
```

```
=====
>>> TEST 1 : Vrification simple des noeuds (srun hostname)
c1
c2

>>> TEST 2 : Test MPI via Python (mpi4py)
MPI SUCCESS: Je suis le rang 0 sur 2, tournant sur le conteneur c1
MPI SUCCESS: Je suis le rang 1 sur 2, tournant sur le conteneur c2

=====
Fin du job : Sun Jan 11 01:33:40 UTC 2026
```

Ce résultat confirme que :

- ▷ SLURM alloue bien deux nœuds distincts (c1 et c2);
- ▷ `srun hostname` exécute une tâche sur chaque nœud;
- ▷ les deux processus MPI sont répartis correctement (rank 0 sur c1, rank 1 sur c2);
- ▷ le job se termine correctement (COMPLETED).

4.5 Export des fichiers vers Windows

Commandes (PowerShell) :

```
docker cp slurmctld:/usr/local/var/mpishare/glcs_slurm/res_1.out .
docker cp slurmctld:/usr/local/var/mpishare/glcs_slurm/res_1.err .
docker cp slurmctld:/usr/local/var/mpishare/glcs_slurm/test-batch.sh .
```

5 Télémétrie : StatsD, InfluxDB et Chronograf

5.1 Envoi d'une métrique StatsD

Depuis le conteneur `slurmctld`, une métrique StatsD a été envoyée sur le port UDP 8125 :

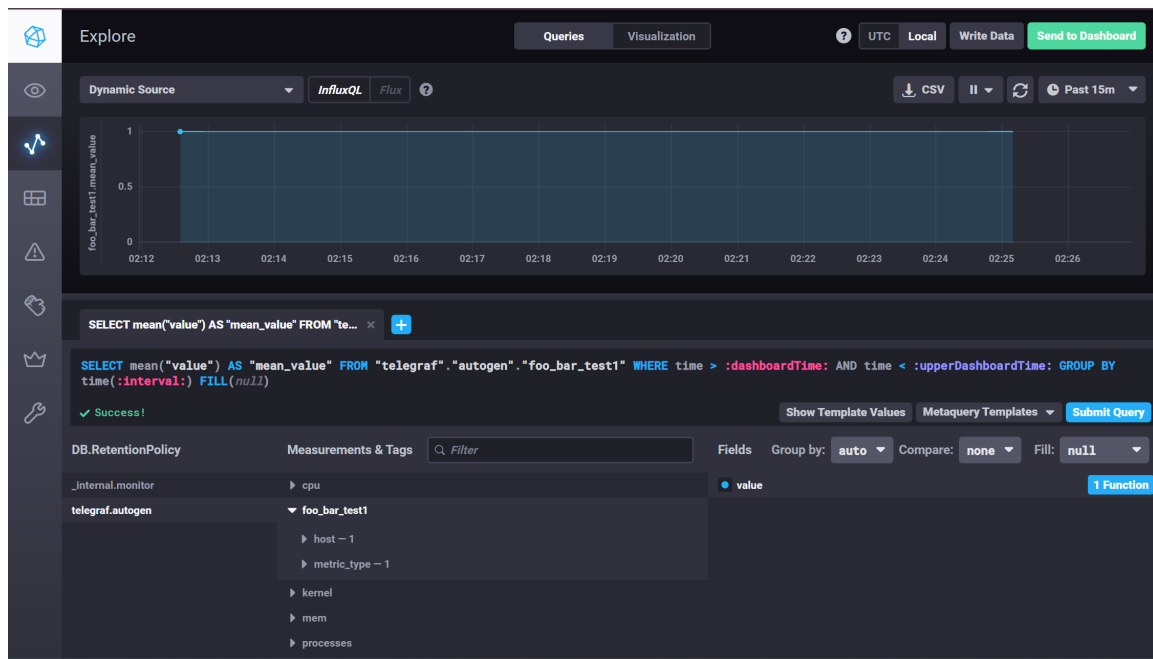
```
echo "foo.bar.test1:+1|g" | nc -u localhost 8125
```

Pour observer une série de points, plusieurs messages peuvent être envoyés :

```
for i in {1..20}; do echo "foo.bar.test1:+1|g" | nc -u localhost 8125; sleep 1;
done
```

5.2 Visualisation dans Chronograf

Dans Chronograf (<http://127.0.0.1:3004>), la métrique est visible dans la base `telegraf.autogen`. La clé `foo.bar.test1` est stockée sous la forme `foo_bar_test1` (les points sont remplacés par des underscores).



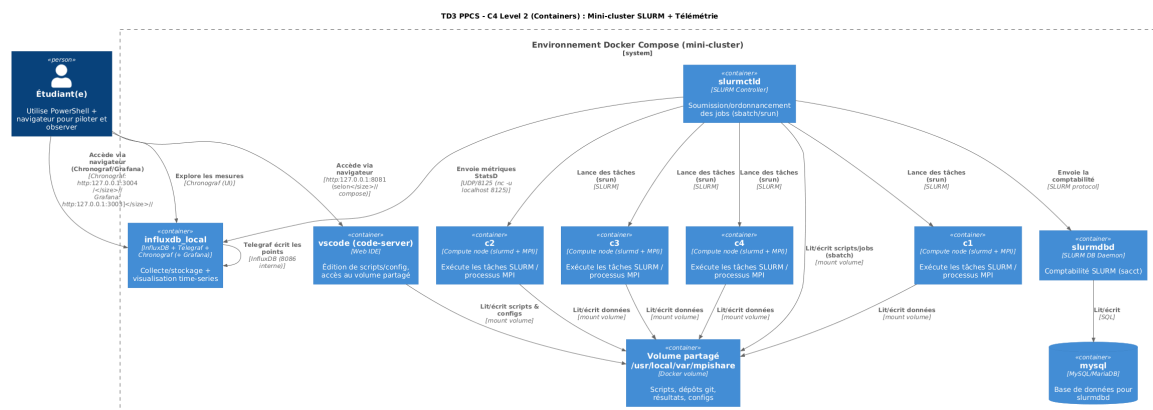
La visualisation confirme la chaîne :

StatsD → Telegraf → InfluxDB → Chronograf

6 Diagramme d'architecture du mini-cluster

Cette section présente le diagramme d'architecture (C4 niveau conteneurs) du mini-cluster SLURM déployé avec Docker Compose.

6.1 Diagramme généré



6.2 Script PlantUML

```
@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml

Person(student, "Etudiant(e)", "Utilise PowerShell et navigateur")
```

```

System_Boundary(s1, "Environnement Docker Compose (mini-cluster)") {
    Container(slurmctld, "slurmctld", "SLURM Controller", "Soumission et
        ordonnanceur des jobs (sbatch, srun)")
    Container(slurmdbd, "slurmdbd", "SLURM DB Daemon", "Comptabilite SLURM (sacct)
        ")
    ContainerDb(mysql, "mysql", "MySQL (MariaDB)", "Base de donnees pour slurmdbd
        ")

    Container(c1, "c1", "Compute node", "slurmd + MPI, execution des taches SLURM
        ")
    Container(c2, "c2", "Compute node", "slurmd + MPI, execution des taches SLURM
        ")
    Container(c3, "c3", "Compute node", "slurmd + MPI, execution des taches SLURM
        ")
    Container(c4, "c4", "Compute node", "slurmd + MPI, execution des taches SLURM
        ")

    Container(influx, "influxdb_local", "InfluxDB + Telegraf + Chronograf", "
        Collecte, stockage et visualisation des metriques")
    Container(vscode, "vscode (code-server)", "Web IDE", "Edition de scripts et
        acces au volume partage")
    Container(volume, "Volume partage", "Docker volume", "/usr/local/var/mpishare
        ")
}

Rel(student, slurmctld, "Soumet et observe", "sbatch, srun")
Rel(student, influx, "Accede via navigateur", "Chronograf (http
    ://127.0.0.1:3004), Grafana (http://127.0.0.1:3003)")
Rel(student, vscode, "Accede via navigateur", "http://127.0.0.1:8081")

Rel(slurmctld, c1, "Lance des taches", "srun")
Rel(slurmctld, c2, "Lance des taches", "srun")
Rel(slurmctld, c3, "Lance des taches", "srun")
Rel(slurmctld, c4, "Lance des taches", "srun")

Rel(slurmctld, slurmdbd, "Envoie la comptabilite", "protocole SLURM")
Rel(slurmdbd, mysql, "Lit et ecrit", "SQL")

Rel(c1, volume, "Lit et ecrit", "mount volume")
Rel(c2, volume, "Lit et ecrit", "mount volume")
Rel(c3, volume, "Lit et ecrit", "mount volume")
Rel(c4, volume, "Lit et ecrit", "mount volume")
Rel(vscode, volume, "Acces scripts et depots", "mount volume")

Rel(slurmctld, influx, "Envoi metriques StatsD", "UDP 8125")

@enduml

```


7 Conclusion

Ce TP3 a permis de déployer un mini-cluster conteneurisé et d'y exécuter des jobs distribués, tout en mettant en place une télémétrie de supervision.

- ▷ L'environnement a été démarré via `docker compose up -d`. Un problème de réseau (`yml_mpinet` manquant) a été corrigé en créant le réseau puis en relançant le déploiement.
- ▷ La configuration SLURM a été validée via `sinfo`, `scontrol` et `sacct`. La synchronisation du fichier `slurm.conf` a permis de disposer de 4 nœuds (`c1..c4`).
- ▷ Un job batch `test-batch.sh` a été soumis via `sbatch`. L'état `COMPLETED` et la sortie `res_1.out` confirment l'exécution MPI distribuée sur `c1` et `c2`.
- ▷ L'envoi de la métrique StatsD `foo.bar.test1` a été observé dans Chronograf sous la forme `foo_bar_test1` dans la base `telegraf.autogen`.

Ce travail illustre l'intérêt de Docker pour reproduire une architecture de type cluster (contrôleur + nœuds), ainsi que l'intérêt d'une chaîne de télémétrie pour observer et superviser l'exécution.